

Algebra Lineal

Computacional

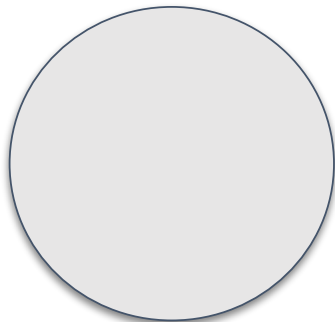
Labo TN



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Nosotros (TN)



Guillermo Marzik (JTP)



Leticia Rodriguez (JTP)



Nicolas Mastropasqua (JTP)



Tomas Chimenti (Ay1)



Tobias Moraut (Ay2)



Tadeo Kaufmann (Ay2)

Última resolución asamblea docente del DC

Reivindicamos la lucha del DC del cuatrimestre pasado, los métodos y la búsqueda de consensos para luchar en unidad. No vamos aceptar que nos sigan atacando.

Última resolución asamblea docente del DC

- Convocar a una asamblea Interclaustrós de Computación y Datos, Miércoles 19/8 17hs en el Aula Magna del Pabellón 1 (en su defecto espacio 0.5)
- Definir un cronograma de manera tal de que todos los días haya al menos una clase curricular en espacio público, y difundirlo de manera continua en las listas, para visibilizar que eso está sucediendo.
- Dar al menos 3 semanas a este periodo de revinculación antes de reevaluar y endurecer las medidas salvo que pase algo excepcional.
- Organizar una nueva comisión encargada de analizar la factibilidad de incorporar cupos a las materias del departamento para los próximos cuatrimestres.

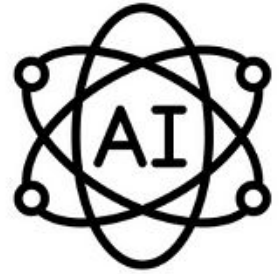
Organización

- Clases Martes de 19hs a 22hs
- 1 TP
- Varios talleres entregables
- 1 ejercicio de labo en parcial

TP

- Grupos de 4 personas. Cada grupo tiene un grupo de referencia
- Sin nota numérica, pero necesarios para aprobar la materia.
- Entregables:
 - Módulo ALC
 - Informe
 - Script con experimentos
 - Defensa
- 1 ejercicio de labo en parcial

Disclaimer: Sobre el uso de la IA



- Copiar y pegar soluciones implementadas sin entenderlas o ser incapaz de reproducirlas tiene un impacto directo en su formación
- Les pedimos que hagan un uso responsable y se aseguren de entender cabalmente cualquier solución que implementen.
- Buscamos que ejerciten su comprensión y capacidad de pensamiento crítico a la hora de leer e interpretar código

Información completa en el campus

“Lineamientos del laboratorio de ALC”

Preguntas que nos hacemos en el labo

¿Cómo dar clases para ~250 personas?

- Asumiendo labos con capacidad de 48 personas
- Teniendo 2 laboratorios asignados

¿Cuánto da $0.1 + 0.2$?

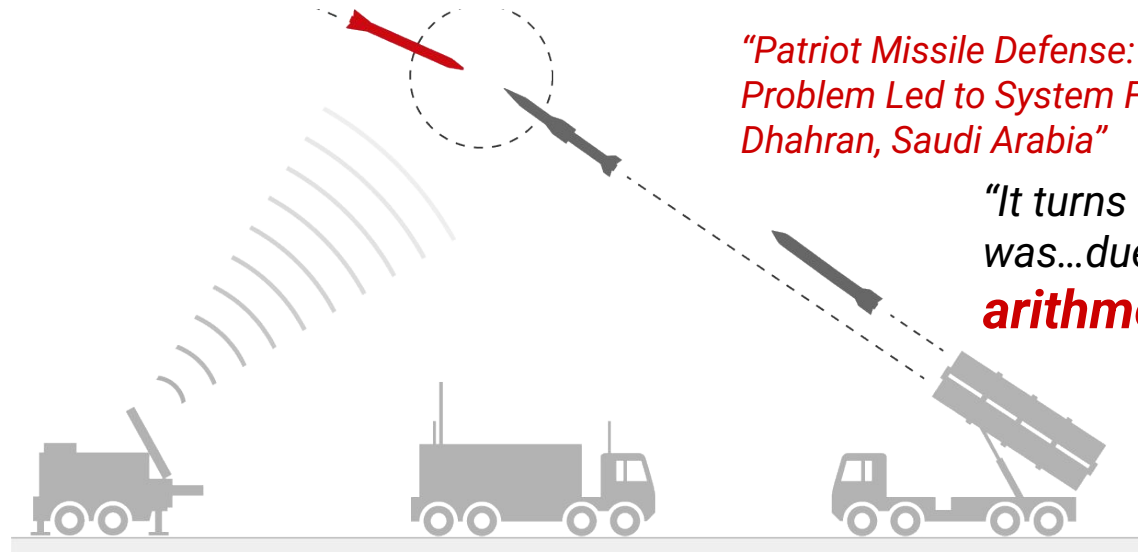
0.3

¿Cuánto da $0.1 + 0.2$ en python?

¿Tanto problema por un error de redondeo?

Problemas numéricos trágicos

El caso del Missile Scud, Saudi Arabia, 1991¹



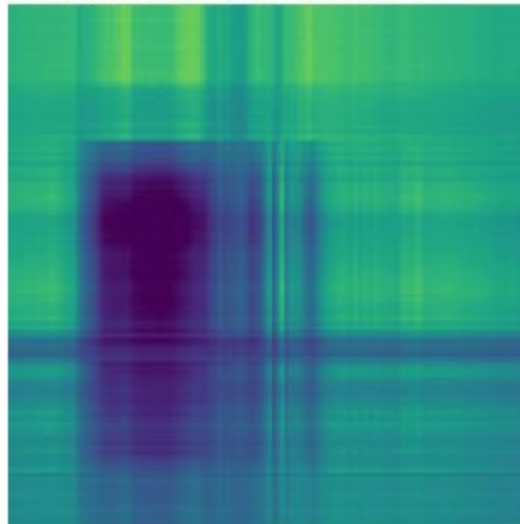
"Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia"

*"It turns out that the cause was...due to computer **arithmetic errors**"*

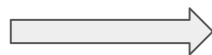
¹<https://www.iro.umontreal.ca/~mignotte/IFT2425/Disasters.html>

¿Por qué está esto en el campus?

Number of vectors = 1



¿Por qué está esto en el campus?



$$X \in \mathbb{N}^{n \times n}$$

Salto de fe (lo veremos más adelante)

$$X = LR$$

Ambas L, R cuadradas

Producto de matrices

$$L = \left[\begin{array}{c|c|c|c} | & | & & | \\ l_1 & l_2 & \cdots & l_n \\ | & | & & | \end{array} \right],$$

$$R = \left[\begin{array}{c|c|c} - & r_1 & - \\ - & r_2 & - \\ & \vdots & \\ - & r_n & - \end{array} \right]$$

Producto de matrices

$$LR = \sum_{i=1}^n l_i r_i$$

Observar la forma de cada sumando, son matrices!

Salto de fe (lo veremos más adelante)

$$X = LR$$

*Plot twist: L y R no son cualquier cosa, si las tomamos de la **descomposición SVD** cada componente puede ordenarse en algún sentido de “importancia”*

Salto de fe (lo veremos más adelante)

X



$\hat{X}_4$



Ejemplo mostrando la suma hasta la cuarta matriz más "importante"

$\hat{X}_8$



$\hat{X}_{16}$



$\hat{X}_{32}$



$\hat{X}_{64}$



*Agregando más componentes
mejoramos la aproximación!*

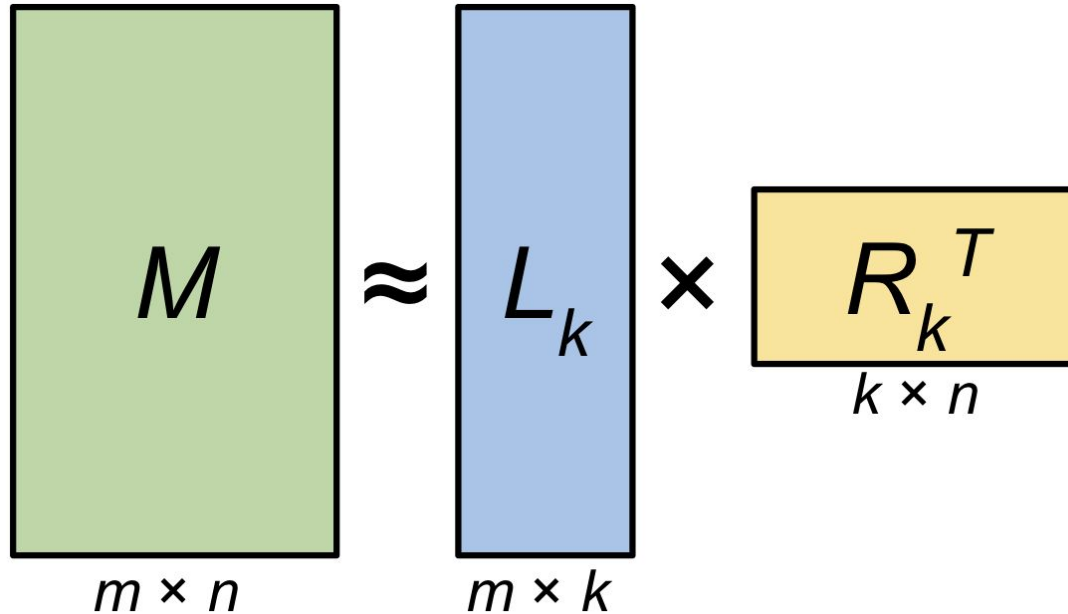


*Es posible que la imagen
tenga $k < n$ componentes no
redundantes (aportan 0
información)*



*Podemos "comprimir" tomando L y R
de $n \times k$ y $k \times n$*

Low rank decomposition

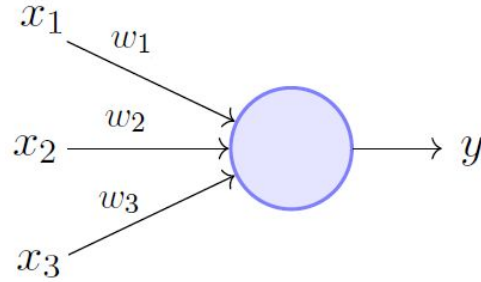


The diagram illustrates the Low Rank Decomposition (LRD) of a matrix M . It shows three matrices: M (green, $m \times n$), L_k (blue, $m \times k$), and R_k^T (yellow, $k \times n$). The matrix M is approximately equal to the product of L_k and R_k^T , represented by the equation $M \approx L_k \times R_k^T$. The dimensions of each matrix are indicated below them: $m \times n$ for M , $m \times k$ for L_k , and $k \times n$ for R_k^T .

$$\begin{matrix} \boxed{M} & \approx & \boxed{L_k} & \times & \boxed{R_k^T} \\ m \times n & & m \times k & & k \times n \end{matrix}$$

Pero k puede ser tan chico como estemos dispuestos a comprimir

¿Pasaron de moda nuestros trucos?

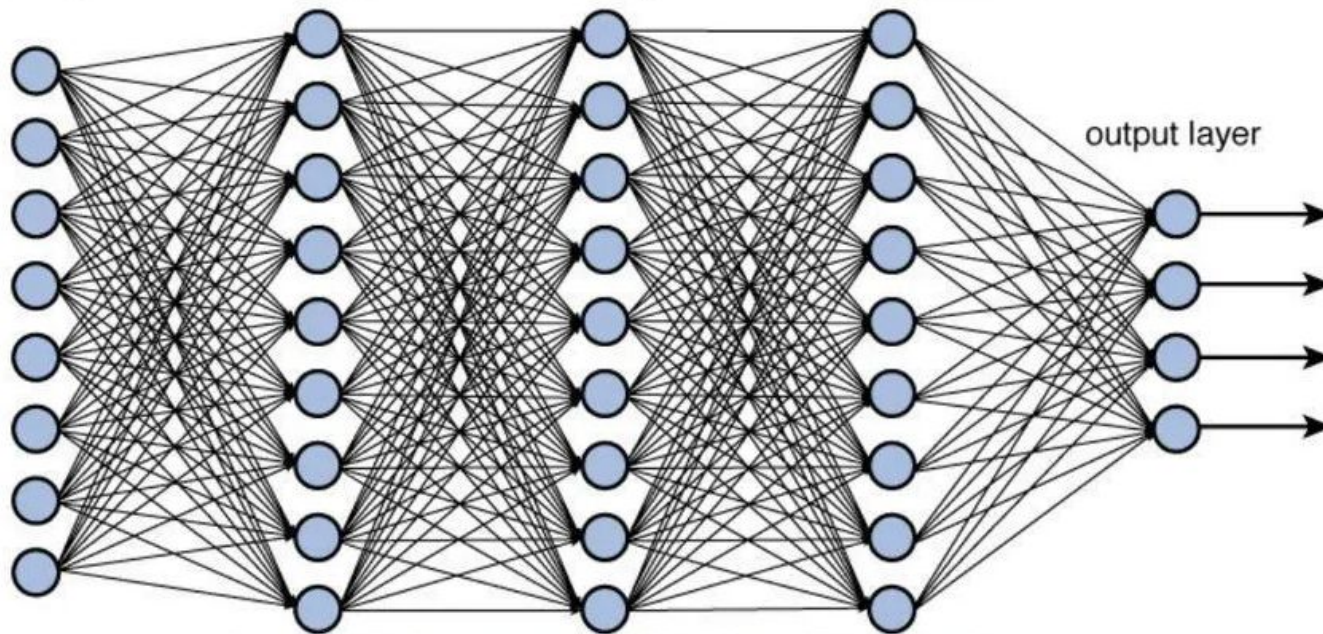


Perceptron Model (Minsky-Papert in 1969)

Algebra lineal en otras partes

$$Y = Wx + b$$

Algebra lineal en otras partes



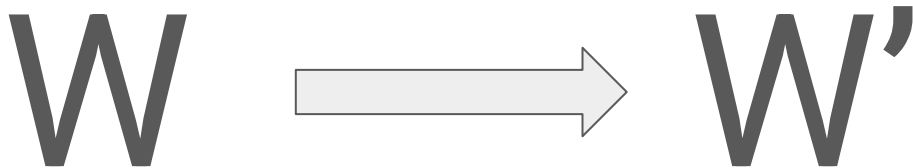
Algebra lineal en otras partes

$$y=Wx+b$$

Entrenar un modelo, “aprender” pesos

W

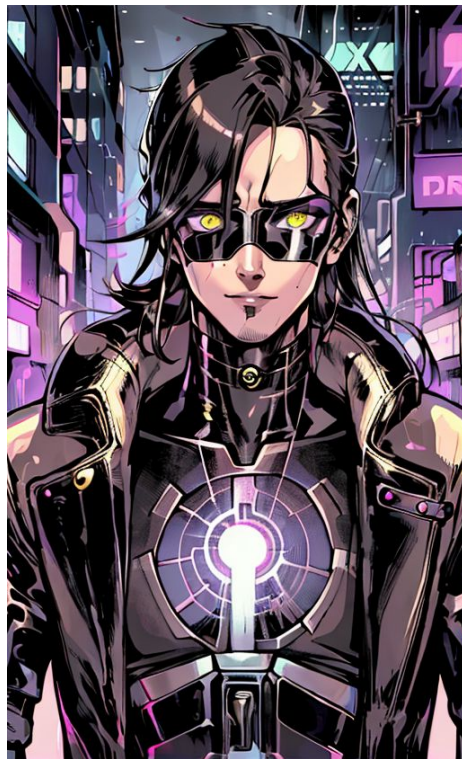
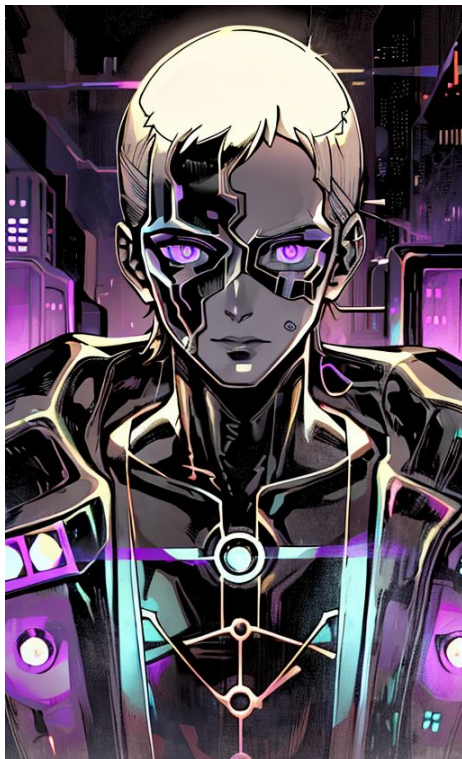
Finetunear un modelo



Finetunear un modelo

$$W' = W + \Delta W$$

¿Finetunear un modelo **gigante**?



Ejemplo, Stable Diffusion

¿Finetunear un modelo gigante?

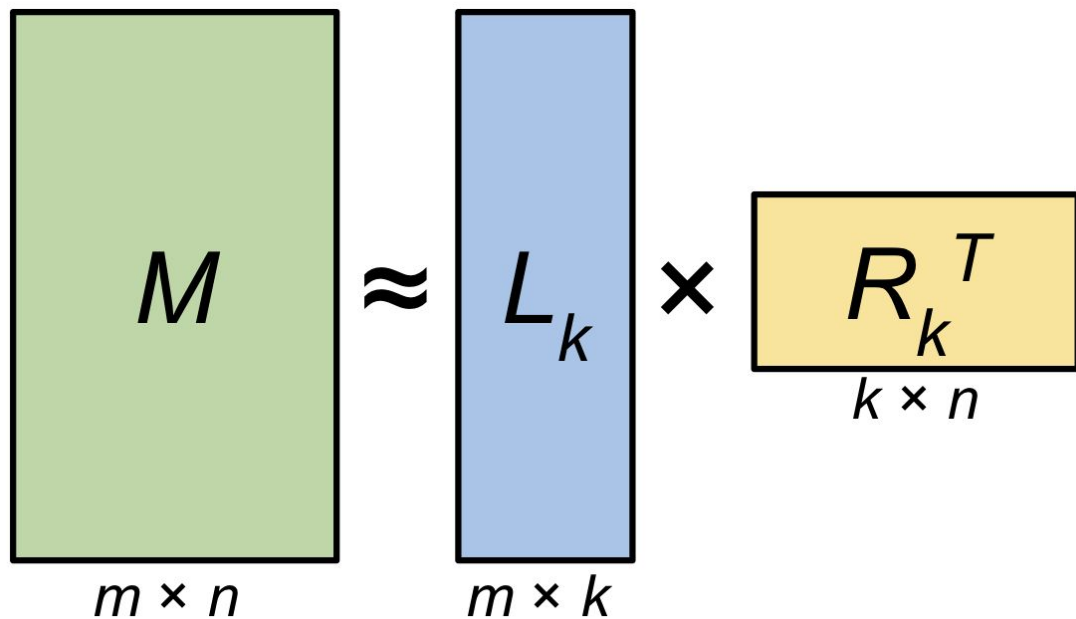
LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

ABSTRACT

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, fine-tuning, which re-trains all model parameters, becomes less feasible. Using GPT-3 175B as an example – deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose **Low-Rank Adaptation**, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to GPT-3 175B fine-tuned with Adam, LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times. LoRA performs on-par or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2, and GPT-3, despite having fewer trainable parameters, a higher training throughput, and, unlike adapters, *no additional inference latency*. We also provide an empirical investigation into rank-deficiency in language model adaptation, which sheds light on the efficacy of LoRA. We release a package that facilitates the integration of LoRA with PyTorch models and provide our implementations and model checkpoints for RoBERTa, DeBERTa, and GPT-2 at <https://github.com/microsoft/LoRA>.

LoRa


$$\begin{array}{ccc} \boxed{M} & \approx & \boxed{L_k} \times \boxed{R_k^T} \\ m \times n & & m \times k \quad k \times n \end{array}$$

Ahora sí: preguntas que nos hacemos en el labo

- Mi algoritmo es **eficiente**?
 - Cómputo
 - Memoria
- Mi algoritmo tiene buenas **propiedades numéricas**?
 - Es numéricamente estable?
 - Converge a una solución?
 - Es preciso?

Mensaje final

VENGAN AL LABO